

Efficient network architecture search via multiobjective particle swarm optimization based on decomposition

Jing Jiang^{a,b}, Fei Han^{a,b,*}, Qinghua Ling^c, Jie Wang^d, Tiange Li^e, Henry Han^e

^a School of Computer Science and Communication Engineering, Jiangsu University, Zhenjiang, Jiangsu 212013, China

^b Jiangsu Key Laboratory of Security Technology for Industrial Cyberspace, Zhenjiang, Jiangsu, 212013, China

^c School of Computer Science, Jiangsu University of Science and Technology, Zhenjiang, Jiangsu 212003, China

^d School of Computer Science and Technology, Anhui University, Hefei, Anhui 230039, China

^e Department of Computer and Information Sciences, Fordham University, NY 48105, USA

ARTICLE INFO

Article history:

Received 19 April 2019

Received in revised form 1 September 2019

Accepted 6 December 2019

Available online 16 December 2019

Keywords:

Convolutional neural network

Neural architecture search

Multiobjective particle swarm optimization

Decomposition

ABSTRACT

The efforts devoted to manually increasing the width and depth of convolutional neural network (CNN) usually require a large amount of time and expertise. It has stimulated a rising demand of neural architecture search (NAS) over these years. However, most popular NAS approaches solely optimize for low prediction error without penalizing high structure complexity. To this end, this paper proposes MOPSO/D-Net, a CNN architecture search method with multiobjective particle swarm optimization based on decomposition (MOPSO/D). The main goal is to reformulate NAS as a multiobjective evolutionary optimization problem, where the optimal architecture is learned by minimizing two conflicting objectives, namely the error rate of classification and number of parameters of the network. Along with the hybrid binary encoding and adaptive penalty-based boundary intersection, an improved MOPSO/D is further proposed to solve the formulated multiobjective NAS and provide diverse tradeoff solutions. Experimental studies verify the effectiveness of MOPSO/D-Net compared with current manual and automated CNN generation methods. The proposed algorithm achieves impressive classification performance with a small number of parameters on each of two benchmark datasets, particularly, 0.4% error rate with 0.16 M params on MNIST and 5.88% error rate with 8.1 M params on CIFAR-10, respectively.

© 2019 Elsevier Ltd. All rights reserved.

1. Introduction

Convolutional neural network (CNN) has led to a series of significant progresses in computer vision society (Lecun, Bengio, & Hinton, 2015; Sainath et al., 2015; Schmidhuber, 2015). It rapidly tackled visual recognition and has increased the top-5 prediction error on ImageNet steadily from 83.6% to 97.75% during the last seven years (Hu, Shen, Albanie, Sun, & Wu, 2019; Krizhevsky, Sutskever, & Hinton, 2012). An important reason for these successive breakthroughs is that numerous fixed but effective CNN architectures have been elaborately designed, such as AlexNet (Krizhevsky et al., 2012), VGGNet (Simonyan & Zisserman, 2015), GoogleNet (Szegedy et al., 2015), ResNet (He, Zhang, Ren, & Sun, 2016), and DenseNet (Huang, Liu, Maaten, & Weinberger, 2017). In this respect, the generic way to strengthen CNN's approximation capability is to make it much deeper and wider (He et al., 2016; Szegedy et al., 2015). Nevertheless, increasing the network capacity might result in the exponential

growth of candidate neural architectures. This no doubt brings a challenge of manually network generation.

Recently, neural architecture search (NAS) has become an emerging research trend since this automated network engineering can easily overcome the aforementioned drawbacks. NAS approaches firstly parameterize the search space with the maximum number of layers and type of operations every layer. And then, they typically specify a search strategy to learn a well-performed architecture (Elsken, Hendrik, & Hutter, 2018; Pham, Guan, & Zoph, 2018). To be specific, Baker et al. presented a meta-modeling algorithm based on reinforcement learning (RL) to automatically design CNN architectures, termed as MetaQNN (Baker, Gupta, Naik, & Raskar, 2017). By combining Q-learning with ϵ -greedy exploration, MetaQNN trains an agent to sequentially determine the type of layers and their hyperparameters. Zoph et al. proposed a NAS method with RL and recurrent network, named NASNet (Zoph & Le, 2017; Zoph, Vasudevan, Shlens, & Le, 2018). Using validation accuracy as the reward signal, the recurrent network as a controller is optimized by policy gradient to generate the best neural architecture description. Zhong et al. proposed a block-wise network generation method using Q-learning with ϵ -greedy exploration, named BlockQNN (Zhong,

* Corresponding author at: School of Computer Science and Communication Engineering, Jiangsu University, Zhenjiang, Jiangsu 212013, China.

E-mail address: hanfei@ujs.edu.cn (F. Han).

Yan, Wu, Shao, & Liu, 2018). BlockQNN represents each convolutional block by a directed acyclic graph (DAG) where every node corresponds to a layer/operation and each directed edge expresses the data flow. The optimal block obtained by the learning agent is finally assembled to stack the whole CNN.

The success gained from neuroevolution of augmenting topologies (NEAT) method (Stanley & Miikkulainen, 2002) has inspired more research interests in evolutionary neural networks (Han, Jiang, Ling, & Su, 2019). It also makes evolutionary algorithm (EA) become a popular choice for realizing NAS (Liu, Simonyan, Vinyals, Fernando, & Kavukcuoglu, 2018; Real et al., 2017; Wang, Sun, Xue, & Zhang, 2018; Xie & Yuille, 2017). Xie and Yuille (2017) proposed GeNet to automatically learn the structures of CNN with genetic algorithm (GA). Each neural architecture as an individual is expressed by a fixed-length binary string that corresponds to connections in a convolutional block. The population is further evolved throughout the genetic process, and the fitness function of each individual is evaluated on validation set (Xie & Yuille, 2017). More recently, Liu et al. (2018) proposed a hierarchical genetic representation with a set of primitive operations in the bottom level. The selected hierarchy cell with the best validation accuracy is finally plugged into a large CNN model for image recognition. Wang et al. (2018) proposed to search the chain-structured CNN by using particle swarm optimization (PSO) along with a novel encoding. In Junior and Yen (2019), combining with variable-length particles and a novel velocity operator, an improved PSO was suggested to search the optimal CNN structure for a given task.

To summarize, the aforementioned works often treat NAS as a single-objective optimization problem. They solely pursue high classification performance but ignore possibly high structure complexity. Thus, the architecture found always contains a large number of redundant operators and connections, such that their efficiency and application scenarios are limited. To this end, Hsu et al. (2018) proposed a multiobjective NAS method (MONAS) based on RL. The reward function is defined as a weighted linear combination of prediction accuracy and power consumption of training. Obviously, formulating such a scalarizing function might incur the loss of many candidate tradeoff solutions as only one solution is provided in a single search run. After that, Tan et al. (2019) further presented an automated mobile NAS (MnasNet), where the original single-objective optimization function was improved by using the customized weighted product. MnasNet is capable of finding multiple Pareto-optimal solutions with considering accuracy and inference latency, and automatically design models for the mobile platform. However, the method also faces the difficulty in determining the value of the weight. More recently, Elsen, Hendrik, and Hutter (2019) proposed an EA-based multiobjective architecture search, in which two objectives termed as accuracy and params are optimized simultaneously. This work breaks through the limitation of scalarizing function and avoids preset parameters, inspiring the usage of multiobjective evolutionary algorithm (MOEA).

Since NAS has two or more objectives that are usually conflicted, it is feasible to reformulate NAS as a multiobjective problem (MOP) and solve the problem by applying MOEAs. MOEAs have been recognized as a popular methodology for approximating the tradeoff solutions undergoing conflicting objectives. They are characterized as population-based metaheuristics for handling MOPs, each of which has more than one Pareto optimal solution. Among MOEAs, decomposition-based methods have gained increasing attention in recent years due to their superior performance with respect to finding tradeoff solutions (Dai, Wang, & Ye, 2015; Zhang & Li, 2007). Generally, they decompose a MOP into several optimization subproblems with a set of reference vectors and optimize the problems based on the steady

neighborhood relation and effective local mating. Because of the simplicity and promising performance of decomposition-based MOEAs, this paper proposes a novel NAS method via multi-objective particle swarm optimization based on decomposition (MOPSO/D), termed as MOPSO/D-Net. The main contributions of this paper are summarized below.

- Considering maximizing the validation accuracy and minimizing the number of learned parameters as two conflicting objectives, the network generation task is first modeled as a MOP. Then, PSO is selected as the basic paradigm for MOEA because it has no complicated evolutionary operators and few parameters compared with other EAs. An improved multiobjective PSO based on decomposition (MOPSO/D) is further presented to obtain Pareto optimal solutions and balance their convergence and diversity.
- A hybrid binary network representation including topology encoding and operation encoding is proposed. The binary string in topology encoding is to represent the network connectivity, while the operation encoding is to express the component layers within a single block. The proposed binary encoding is inspired by the work in Xie and Yuille (2017), however, it encourages the emergence of various kinds of operators in candidate network architectures.
- Extensive experimental studies show that the proposed algorithm eventually maintains numerous tradeoff solutions in comparison with single-objective NAS methods. Moreover, MOPSO/D-Net also provides competitive results on two benchmark datasets, particularly, 0.4% error rate with 0.16M params on MNIST and 5.88% error rate with 8.1M params on CIFAR-10, respectively.

The rest of this paper is organized as follows. Section 2 briefly introduces some basic concepts related to this study. Section 3 presents the proposed MOPSO/D-Net for CNN automating engineering. Experimental setup and results are shown in Section 4, followed by the conclusions in Section 5.

2. Preliminaries

2.1. Basic concepts

Generally, a MOP can be stated as

$$\begin{aligned} \min \quad & F(x) = (f_1(x), f_2(x), \dots, f_m(x))^T \\ \text{s.t.} \quad & x \in \Omega \end{aligned} \quad (1)$$

where Ω is the *decision space* and F consists of m objective functions, respectively. $\{F(x) | x \in \Omega\}$ is the *attainable objective set*.

Considering two decision vectors $x_A, x_B \in \Omega$, the vector x_A is said to *dominate* x_B (denoted by $x_A \prec x_B$) if and only if $\forall i : f_i(x_A) \leq f_i(x_B) \wedge \exists j : f_j(x_A) < f_j(x_B)$, where $i, j \in \{1, \dots, m\}$. Moreover, a solution x^* is *Pareto optimal*, if there is no other solution $x \in \Omega$ such that $x \prec x^*$. The set of all the Pareto optimal solutions is defined as Pareto set (PS) and its image is called Pareto front (PF). The *ideal point* is another important concept for illustrating the shape of PF. It is a vector $z^* = (z_1^*, \dots, z_m^*)^T$, where $z_i^* = \min_{1 \leq i \leq m} \{f_i(x) | x \in \Omega\}$.

2.2. Particle swarm optimization

Particle swarm optimization is a population-based stochastic optimization method, in which a number of particles are used to traverse the space (Kennedy & Eberhart, 1995). Each particle is a candidate solution in the space. To find out the global optimal (referred to as the global best position in PSO), each particle is updated at generation t as follows:

$$\begin{aligned} v_i(t+1) &= v_i(t) + c_1 r_1(t)(p_i(t) - x_i(t)) + c_2 r_2(t)(p_g(t) - x_i(t)) \\ x_i(t+1) &= x_i(t) + v_i(t+1) \end{aligned} \quad (2)$$

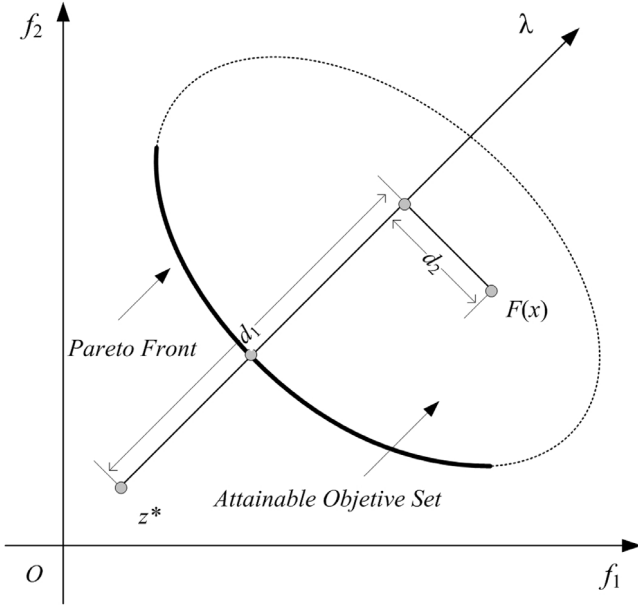


Fig. 1. Illustration of the PBI approach.

where $v_i(t)$ and $x_i(t)$ ($1 \leq i \leq N$) are the velocity and position of i th particle at generation t ; $p_i(t)$ is the personal best position of i th particle ($pbest_i$) and $p_g(t)$ is the global best position of swarm ($gbest$); c_1 and c_2 are two positive acceleration constants; $r_1(t), r_2(t) \in [0, 1]^n$ are two random vectors.

2.3. The penalty-based boundary intersection

Commonly used decomposition methods are weighted sum (WS), Tchebycheff (TCH) and PBI, each of which decomposes a MOP into multiple scalar subproblems to be optimized. Among these approaches, PBI employs reference vectors and a penalty parameter θ for minimizing both the distance d_1 and d_2 , which are used to measure the convergence and diversity, respectively (see Fig. 1). The scalarizing subproblem optimization is formulated as

$$\begin{aligned} \min \quad & g(x|\lambda_i, z^*) = d_1 + \theta d_2 \\ \text{s.t.} \quad & x \in \Omega \end{aligned} \quad (3)$$

where $d_1 = \frac{\|(F(x) - z^*)^T \lambda_i\|}{\|\lambda_i\|}$ and $d_2 = \|(F(x) - z^*) - d_1 \frac{\lambda_i}{\|\lambda_i\|}\|$. The reference vector is a vector $\lambda_i = (\lambda_i^1, \dots, \lambda_i^m)^T$, where $\lambda_i^j \geq 0$ for all $j = 1, \dots, m$, and $\sum_{j=1}^m \lambda_i^j = 1$.

3. The proposed approach

3.1. Multiobjective NAS formulation

Conventional approaches conducted NAS by optimizing such an optimization problem under some constraints

$$\begin{aligned} \min \quad & f(x) \\ \text{s.t.} \quad & g(x) \leq T \end{aligned} \quad (4)$$

where $f(x)$ is the prediction error of neural architecture; $g(x)$ is the number of parameters, the inference latency or FLOPS; T is the target value of params/latency/FLOPS. However, this formulation only optimizes for a single metric. The obtained solution could not reach the compromise between accuracy and efficiency (Tan et al., 2019). To this end, both the validation

accuracy and params are considered as two objectives simultaneously in this study. The multiobjective NAS formulation is then constructed as follows

$$\begin{aligned} \min \quad & F(x) = (f_1(x), f_2(x))^T \\ \text{s.t.} \quad & x \in \Omega \end{aligned} \quad (5)$$

where $f_1(\cdot)$ and $f_2(\cdot)$ are two fitness functions originated from the evaluation criteria. Intuitively, maximizing the classification accuracy of CNN is to enhance its generalization ability, while minimizing the params is to reduce its computation cost. These two goals conflict with each other in some cases, that is, the optimal performance on one objective often results in a low performance on another objective. Thus, these two metrics are very suitable for defining $f_1(\cdot)$ and $f_2(\cdot)$, which creates the necessity of a compromise to be reached (Mukhopadhyay, Maulik, Bandyopadhyay, & Coello Coello, 2014).

3.2. The hybrid binary network representation

The handcrafted architectures are usually built by repeated convolutional blocks (Elsken et al., 2019). Thus, this study is mainly proposed to search for the optimal block rather than an entire network. The proposed binary network representation is to encode each convolutional block into a binary string, by which the decision vector and search space are specified. Given the i th convolutional block B_i ($i = 1, \dots, N$) in the population, it is represented as $x_i = (x_{i,T}, x_{i,O})$, where $x_{i,T}$ and $x_{i,O}$ are topology encoding and operation encoding, respectively.

3.2.1. Topology encoding

The idea of topology encoding is inspired by GeNet (Xie & Yuille, 2017). Likewise, a binary string is used to represent the connections among L nodes in a single architecture. All the nodes are arranged in order and the data flow is only allowed from a lower-numbered node to higher-numbered nodes. Every node except for the first one has its binary substring with the length of $l-1$, where l is the index number of the l th node. The connection bit between the j th node and k th node is denoted as x_k^j . Hence,

$$x_k^j = \begin{cases} 1 & \text{if node } j \text{ is connected to node } k \\ 0 & \text{otherwise} \end{cases} \quad (6)$$

where $k = 2, 3, \dots, L$ and $j = 1, 2, \dots, k-1$.

The length of $x_{i,T}$ is calculated as $1+2+\dots+(L-1) = L(L-1)/2$. Naturally, the following points are of great importance to make topology encoding $x_{i,T}$ work. Firstly, the redundancy node that has neither predecessor nor successor should be excluded from the block. Secondly, the previous layer of a block is required to send its information to all the intermediate nodes that have no predecessor, and the succeeding layer should receive the output from every intermediate node without a successor. Thirdly, since a single node l might receive more than one input, the destination node l should take every input as a part of information aggregation. Fig. 2 shows three examples of binary topology encoding in the case of $L = 5$.

3.2.2. Operation encoding

The operation encoding is further proposed to represent component layers within blocks. Compared to GeNet, the operation encoding encourages the use of multiple kinds of operators, rather than only utilizing 3×3 convolutions within a single convolutional block. Inspired by current handcrafted architectures, eight primitive operations are considered in this study. The details and the corresponding binary substrings are shown in Table 1. As observed in Table 1, every operation is encoded by a 3-bit substring, and thus the operation encoding $x_{i,O}$ of B_i is formed by L binary substrings with the length of $3L$.

Table 1
Details of the operation encoding scheme.

Type of operations	Stride	Padding	Binary substring
1×1 max pooling	1	0	000
1×1 convolution	1	0	001
3×3 average pooling	1	1	010
3×3 max pooling	1	1	011
3×3 convolution	1	1	100
5×5 convolution	1	2	101
1×3 then 3×1 convolution	1	[0,1], [1,0]	110
1×5 then 5×1 convolution	1	[0,2], [2,0]	111

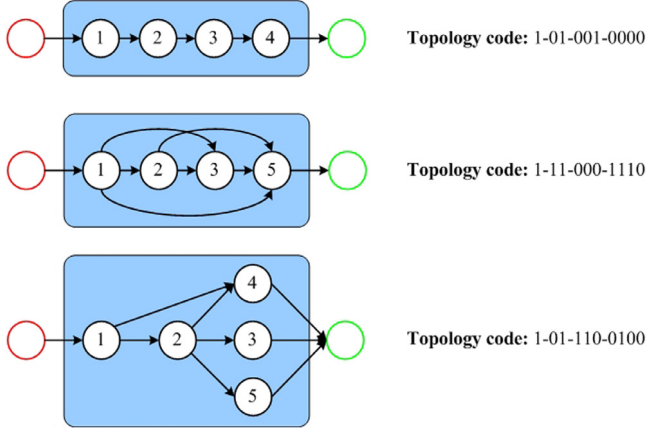


Fig. 2. Examples of the network blocks with different topology encodings ($L = 5$).

Closer inspection of the table shows that only the filters with small receptive field are used, like 1×1 and 3×3 convolutions. This way follows the insights from Simonyan and Zisserman (2015) and Szegedy, Ioffe, and Vanhoucke (2016), which stated that a large receptive field can be obtained by stacking filters with small kernel size. Besides, all the convolutions share the same number of channels K as wider convolution can be obtained by depth-wise concatenation of multiple filters. To make the width and height of the information flow remain unchangeable, the stride is fixed as 1 with or without padding for every node. Additionally, batch normalization is adopted after each convolution and before ReLU activation to further reduce the search space.

Overall, given L nodes in $\mathcal{B}_i$, the size of search space Ω of MOPSO/D-Net is explicitly determined as

$$\Omega = \Omega_T \times \Omega_O = 2^{L(L-1)/2} \times 2^{3L} \quad (7)$$

where Ω_T is the search space for topology encoding, and Ω_O is for operation encoding.

3.3. The proposed MOPSO/D for CNN architecture search

Unlike single-objective NAS approaches, we develop MOPSO/D as the search strategy for discovering the tradeoff solutions with two conflicting objectives. MOPSO/D firstly decomposes the formulated multiobjective NAS into N scalar subproblems and optimizes them simultaneously based on an adaptive PBI approach. In this subsection, we firstly analyze the influence of the value of penalty θ in PBI and propose an improved PBI with adaptive penalty adjustment strategy, termed as adaptive PBI. Then, the framework of the proposed MOPSO/D-Net is given based on MOPSO/D.

3.3.1. The adaptive PBI method

Existing studies have shown that the penalty controls the balance between convergence and diversity of solutions (Ming,

Wang, Zha, & Zhang, 2017; Sheng, Jiang, & Jiang, 2017). A small θ emphasizes the convergence since it accepts the solution that is far away from the reference vector but much closer to the PF, whereas a large θ prefers the diversity as it encourages solutions to approach the uniformly distributed reference vectors. Therefore, the search performance of PBI-based MOEAs is highly related to the value of θ .

Nevertheless, a fixed θ along with uniformly distributed vectors may encounter the difficulty in approaching the complicated Pareto front of real-world problems. The viewpoint is supported by Fig. 3, where the bold line in each subfigure depicts a PF with sharp peak and long tail. The solutions A–D are associated with the reference vectors λ_1 to λ_4 , and the solid lines show two different contour lines and landscapes of PBI with $\theta = 1$ and $\theta = 5$. In Fig. 3(a), since A and B are the intersection points between the PF and reference vectors, they should be the optimal solutions to λ_1 and λ_2 , respectively (Qi et al., 2014). However, the solution A will be replaced by B as $g(A|\lambda_1, z^*) > g(B|\lambda_1, z^*)$ observed from the contour line. In this case, an inappropriate θ makes an optimal solution near the reference vector to be replaced by another solution near the ideal point z^* . Thus, the diversity of solutions is degenerated as those points reside in the boundary regions may finally gather in the intermediate parts.

Regarding Fig. 3(a), a feasible way to enhance the diversity is to increase θ , such that the solutions that tend to deviate the reference direction will be drastically penalized. Note that, increasing θ may raise another problem, which is briefly illustrated in Fig. 3(b). The solution D is intuitively viewed as a better one compared to solution C although D has a larger d_2 . However, C will not be replaced by D in this case since $g(C|\lambda_1, z^*) < g(D|\lambda_1, z^*)$ according to the contour line. It is obvious that the convergence speed is therefore impacted due to a large θ .

Moreover, boundary solutions are nontrivial in this study. These solutions often represent the convolutional blocks with higher classification performances or lower params. An inappropriate θ might incur a loss of these important boundary solutions. As a result, an adaptive penalty adjustment strategy is proposed as shown in Eq. (8).

$$\theta_i = \theta_{min} + (\theta_{max} - \theta_{min}) \times \frac{t}{maxgen} \times (\max_{1 \leq j \leq m} \lambda_i^j - \min_{1 \leq j \leq m} \lambda_i^j) \quad (8)$$

where t is the current generation number and $maxgen$ is a pre-defined maximum of t . As shown in Eq. (8), θ will be iteratively updated throughout the search process. At the early stage, the search algorithm prefers a fast convergence speed with a smaller θ . With increasing θ , it might prefer well distributed solutions. Besides, each subproblem also has its own θ_i , i.e., the boundary subproblems are constrained by larger penalties while intermediate subproblems are with smaller penalties in pursuit of a fast convergence. In a word, the adaptive PBI is proposed to increase the selection pressure for exploring the boundary regions. It can also help to better control the balance between the convergence and diversity when addressing Eq. (5) with an unknown PF geometric.

3.3.2. Framework of MOPSO/D-Net

To summarize, the pseudo-code of MOPSO/D-Net is given in Algorithm 1. The complete framework mainly consists of two parts, namely, the initialization and main loop. Several remarks on the algorithm are listed below.

(1) In the initialization step, a particle swarm P is randomly initialized at first (line 3). The initial velocity of each particle is set as 0. The position is determined by topology encoding and operation encoding, where every bit is independently sampled from Bernoulli distribution $B(0.5)$. After that, N reference vectors are uniformly generated to convert multiobjective NAS into several

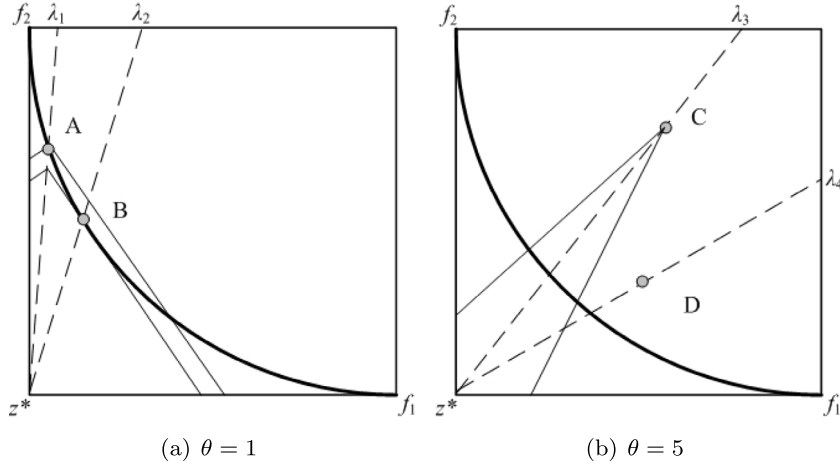


Fig. 3. The influence of the penalty θ on the search performance.

Algorithm 1 MOPSO/D-Net

Input: N (the swarm size), T_a (age threshold of subproblems), t (generation index)
Output: P (final solutions)

```

1: /*Initialization*/
2:  $P \leftarrow \text{RandomInitialize}(N)$ ;
3:  $\lambda \leftarrow \text{UniformPoint}(N)$ ;
4:  $z^* \leftarrow \text{ReferencePoint}(P)$ ;
5:  $\text{Age} \leftarrow \text{ZeroInitialize}(N)$ ;
6: /*Main Loop*/
7:  $t \leftarrow 0$ ;
8: while  $t < \text{maxgen}$  do
9:    $[P', P''] \leftarrow \text{SwarmDivision}(P, \text{Age}, T_a)$ ;
10:   $P' \leftarrow \text{ParticleUpdate}(P')$ ;
11:   $P'' \leftarrow \text{ParticleReinitialization}(P'')$ ;
12:   $z^* \leftarrow \text{ReferencePoint}(P' \cup P'', z^*)$ ;
13:   $\theta \leftarrow \text{AdaptivePBI}(\lambda, t)$ ;
14:   $P \leftarrow \text{EnvironmentalSelection}(P \cup P' \cup P'', \lambda, z^*, \theta)$ ;
15:   $\text{Age} \leftarrow \text{UpdateAge}(P, P' \cup P'', \text{Age})$ ;
16:   $t \leftarrow t + 1$ ;
17: end while
18: return  $P$ 

```

subproblems, each of which is optimized with a certain particle (line 4). In line 5, the reference point z^* is obtained by evaluating the initial swarm. In line 6, the age of each particle, counting for the number of generations it has not been changed, is initialized as 0.

(2) The swarm is divided into two portions updated upon distinct rules (line 10). The particle whose age is lower than the threshold T_a is updated by Eq. (2), and its corresponding convolutional block is trained by Adam or SGD optimizer (line 11). Otherwise, the particle should be reinitialized as follows.

$$x_i(t+1) = \frac{p_{g,i}(t) - p_i(t)}{2} + r(t) \times |p_{g,i}(t) - p_i(t)| \quad (9)$$

where $p_i(t)$ and $p_{g,i}(t)$ are the positions of personal best and global best particle to the i th subproblem; $r(t) \in [0, 1]^n$ is a random vector. The re-initialization strategy (line 12) is adopted when a particle has no improvement within predefined T_a generations. In this way, the probability of premature convergence might be decreased.

(3) The penalty of each subproblem is updated by Eq. (8) (line 14). After that, an environmental selection (lines 15) is suggested

and the pseudo-code is introduced in Algorithm 2. Every subproblem keeps only one particle that has the best scalarizing value among the current swarm P , ensuring the most suitable particles finally survive from the selection.

(4) At the end of each generation, the age is set to 0 if the subproblem has been improved within the generation (lines 16). Otherwise, the age of subproblem is increased by 1.

Algorithm 2 EnvironmentalSelection(P, λ, z^*, θ)

Input: P (current swarm), λ (reference vectors), z^* (reference point), θ (penalty)
Output: Q (new swarm)

```

1:  $Q \leftarrow \emptyset$ ;
2: for  $i = 1$  to  $|\lambda|$  do
3:    $Q \leftarrow Q \cup \{x_j | \min_{x_j \in P} g(x_j | \lambda_i, z^*, \theta)\}$ 
4:    $P = P \setminus \{x_j\}$ 
5: end for
6: return  $Q$ 

```

4. Experiments

4.1. Experimental setup

Since MOPSO/D-Net updates the particles in parallel within a single generation, it motivates us to employ a distributed asynchronous system to accelerate the search. The implementation makes the search cost remain within an acceptable range. We use a distributed asynchronous system equipped with a single Nvidia P100 GPU to carry out the experimental simulations on MNIST and CIFAR-10 datasets (see Fig. 4). The system has a major computing node with 20 individual workers. The major node conducts the algorithm and shares its memory with all the workers. Each worker is responsible for training a single convolutional block and uploading its result to the major node in every generation.

4.2. Experiments on MNIST dataset

Experimental simulations are first conducted on the MNIST dataset (LeCun, Bottou, Bengio, & Haffner, 1998), which contains 60,000 handwritten digit images for training and 10,000 images for testing. We further separate 10,000 images as validation data from the training set. The validation set provides the error rate and params as the particle's fitness once the training process of a worker is completed.

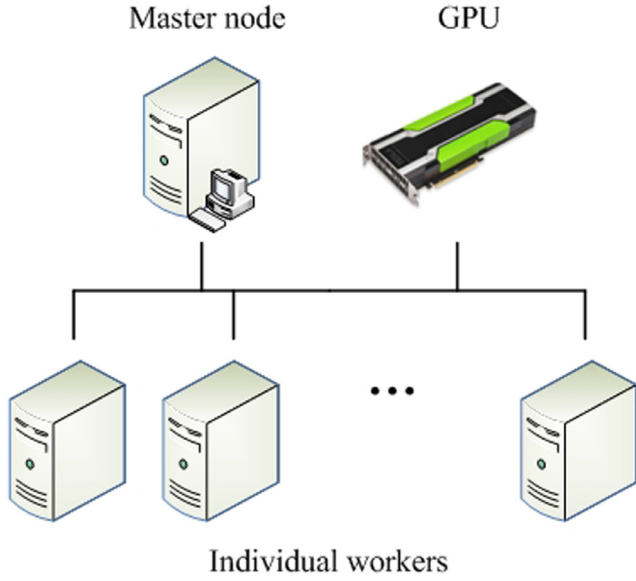
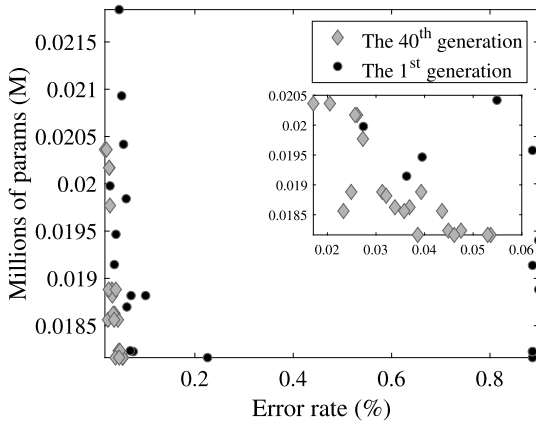


Fig. 4. The distributed asynchronous system with a master node and several individual workers.

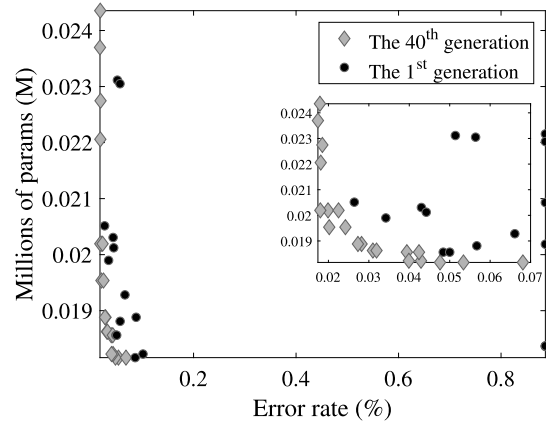
As for each convolutional block, K denotes the number of convolutional kernels. It is fixed as $2^3 = 8$ across the evolution. On the other hand, L represents the number of nodes in a single block. It determines the dimension of decision variable and the size of search space. In this study, four candidate values ($L \in \{3, 4, 5, 6\}$) are adopted on MNIST to investigate the effect of L . The dimensions of corresponding decision variables are thereby calculated as 12, 18, 25 and 33, respectively.

The workers in the distributed asynchronous system train each network for 6 epochs, using SGD optimizer with a momentum of 0.9 and weight decay of 0.0005. The learning rate is initialized as 0.1 and is multiplied by 0.1 after 5 epochs. In respect of MOPSO/D, the size of swarm N is set to 20, and the algorithm runs over 40 generations. Thus, there are 800 architectures evaluated over the evolution in total. Other relevant parameters are listed as follows: $c_1 = 1.4$, $c_2 = 1.4$, $T_a = 2$, $\theta_{min} = 1$ and $\theta_{max} = 10$.

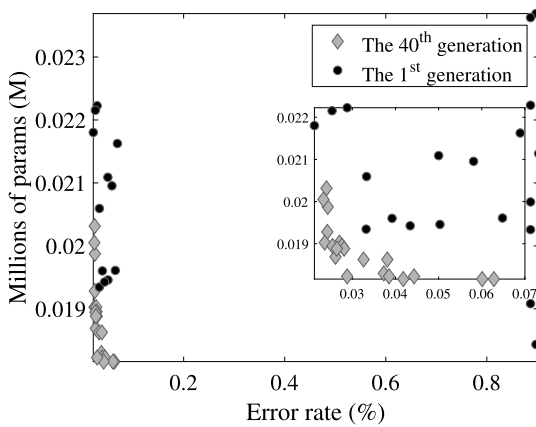
Fig. 5 exhibits the Pareto optimal solutions obtained by MOPSO/D-Net across 40 rounds. It is apparent that the swarm in each subfigure consistently approximates the estimated ideal point z^* during the generations. Most tradeoff solutions eventually achieve relatively smaller validation error rates and fewer params compared with those in the first generation. It is mainly attributed to the search process undergoing two objectives. During the process, those solutions away from the reference vector or with large scalarizing values are discarded. The fact verifies the efficacy of formulated multiobjective NAS and the proposed MOPSO/D-Net. Moreover, due to the adaptive PBI strategy and



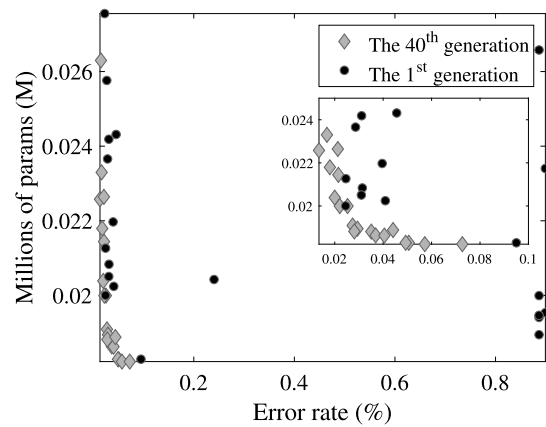
(a) $L = 3$



(b) $L = 4$



(c) $L = 5$



(d) $L = 6$

Fig. 5. The initial and final particles obtained by MOPSO/D-Net with the MNIST dataset.

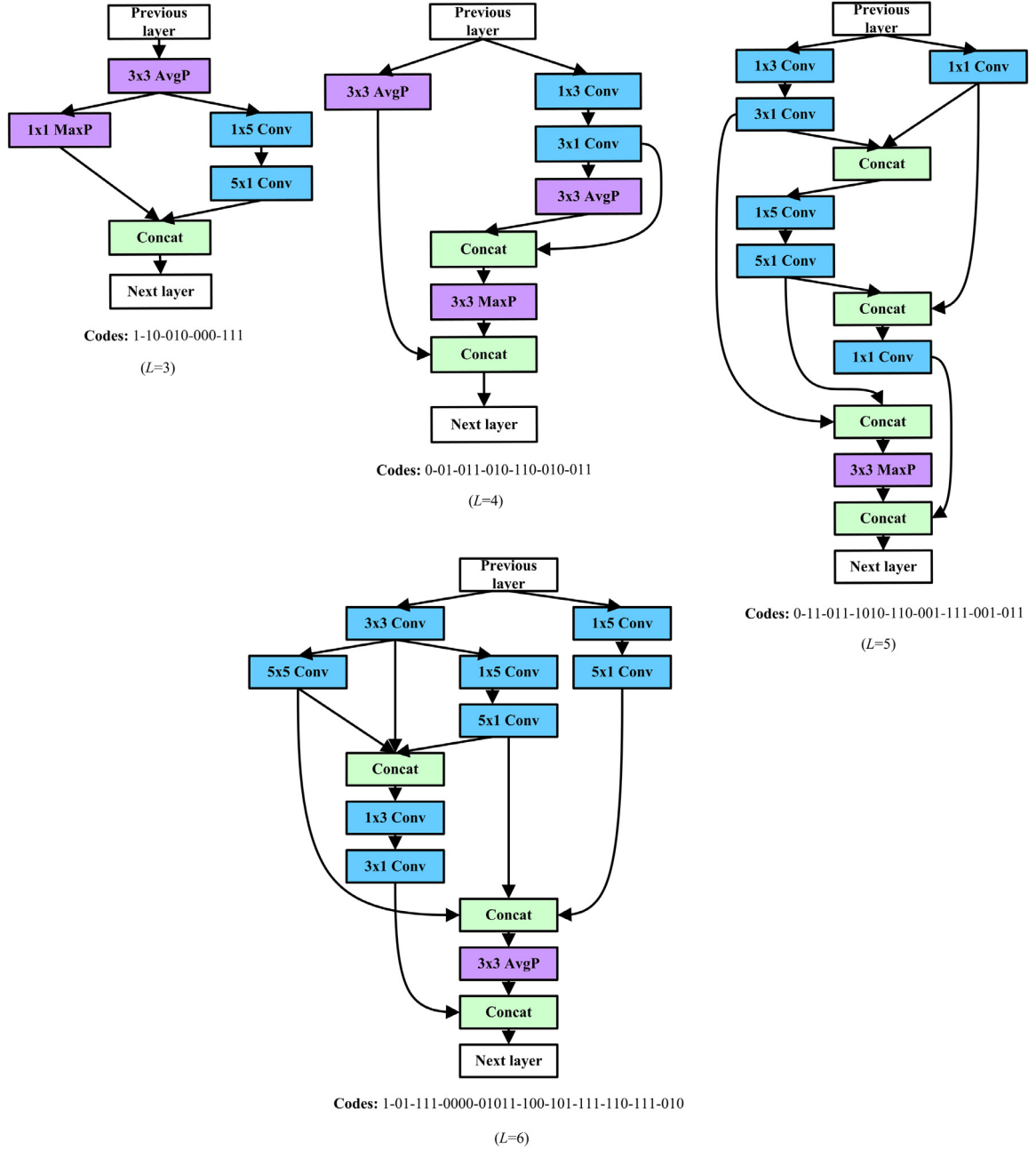


Fig. 6. The selected convolutional blocks with the MNIST dataset.

uniformly reference vectors, the final solutions have a good diversity over the objective space. It avoids the situation of missing boundary solutions, such that more candidates have been maintained throughout evolution.

Since multiple solutions have been yielded by MOPSO/D-Net over one single run, the necessity of selecting one from the swarm should be highlighted. Based on the *prior* knowledge, we choose the block with the best classification performance among the solutions. Fig. 6 further compares the convolutional blocks selected from four populations of solutions. It is worth noting that 1×3 then 3×1 and 1×5 then 5×1 convolutions are more frequently seen in all cases. Besides that, the convolution operators are always followed by pooling operators in the latter two cases. These facts reveal the advantage of the proposed algorithm on restraining the complexity of evolved blocks. The selected blocks also reduce the likelihood of over-parameterized CNN.

Concerning the full training, we transform every simple convolutional block into an entire network through repeating blocks. The number of kernels to each convolution operator is fixed as $2^5 = 32$. Therefore, four CNNs are established in this way. These architectures are trained by SGD with a momentum of 0.9 and weight decay of 0.0005. The training phase has 20 epochs with a learning rate of 0.01, 5 epochs with a learning rate of 0.001, and 2 epochs with a learning rate of 0.0001. Fig. 7 compares the performances among four established CNNs in terms of error rate and params. The empirical result indicates that the auto-generated CNN of the case $L = 6$ achieves the best performance, with 0.4% testing error and 0.16 M parameters on MNIST.

Next, the CNN found is compared with current manual and automated models in Table 2. As observed from the table, the testing accuracy of MOPSO/D-Net firstly surpasses the baseline LeNet-5 and ResNet. It also approximates the state-of-the-art results of CapsNet (Sabour et al., 2017) and DropConnect (Lee et al.,

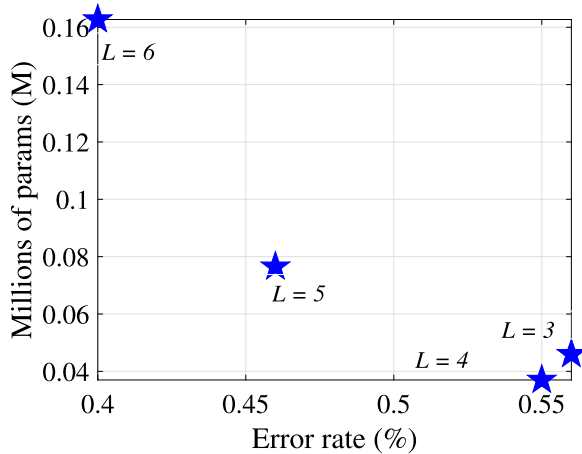
Table 2

Results comparison among the proposed MOPSO/D-Net and peer algorithms on the MNIST dataset.

	Search manner	Error rate (%)	Params (M)	Inference latency (ms)	Search cost
LeNet-5 (LeCun et al., 1998)	Manual	0.75	0.43	–	–
ResNet (Chen, Rubanova, Bettencourt, & Duvenaud, 2018)		0.41	–	–	–
CapsNet (Sabour, Frosst, & Hinton, 2017)		0.25	–	–	–
DropConnect (Lee, Gallagher, & Tu, 2013)		0.21	–	–	–
MnasNet (Tan et al., 2019)	Automated	0.82	3.5	476	1 GPU / 3.9 hours
IPPSO (Wang et al., 2018)		1.21	1.47	206	2 GPUs / 2.5 hours
psoCNN (Junior & Yen, 2019)		0.44	3.26	449	1 GPU / 7.3 hours
GeNet (Xie & Yuille, 2017)		0.38	–	–	10 GPUs / 2 days
MOPSO/D-Net		0.40	0.16	271	1 GPU / 6 hours

† Inference latencies are given by conducting the learned models on our experimental environment while search costs are cited from the original references, respectively.

§ All the indices of MnasNet are obtained by reproducing the algorithm.

**Fig. 7.** The comparison among CNNs with different architectures on the MNIST dataset.

2013), both of which greatly benefit from the special techniques that generally excluded in the candidate architectures of NAS approaches. Apart from the handcrafted models, MOPSO/D-Net outperforms three automated models from IPPSO (Wang et al., 2018), psoCNN (Junior & Yen, 2019) and MnasNet (Tan et al., 2019). It also obtains a similar error rate with GeNet (Xie & Yuille, 2017). Regarding the structural complexity, the proposed algorithm finally yields a more compact and efficient CNN. It has 0.16M params, which are $2\times$ fewer than LeNet-5, $9\times$ fewer than IPPSO, $20\times$ fewer than psoCNN and $21\times$ fewer than MnasNet. The inference latency is also provided by Table 2. The CNN obtained by MOPSO/D-Net requires 271 ms for inference all the testing images. It is higher than IPPSO, but slightly lower than MnasNet and psoCNN. The evidence suggests that MOPSO/D-Net can create a competitive CNN architecture on MNIST, in comparison with both manually-designed and NAS approaches. This highlights the superiority of the usage of multiobjective NAS fashion.

Moreover, GeNet requires expensive computation resources (more than 24 hours with multiple GPUs). This phenomenon may be explained by the fact that it attempts to search for a full architecture. Despite the high cost, we notice that learning an entire topology could build a better CNN than MOPSO/D-Net. It is argued that solely optimizing for a near-optimal block might lead to a small sacrificing accuracy. On the other hand, IPPSO and psoCNN require fewer computation resources as they can accomplish the task within 2.5 hours on 2 GPUs and 7.3 hours on 1 GPU, respectively. These two methods suggest automating chain-structured and shallow architecture, resulting in much easier training and more quickly fitness evaluation. Compared with

two peer algorithms, MOPSO/D-Net allows to build both chain-structured and multi-branch architectures, and further reduces the search time down to an acceptable level, i.e., 6 hours on a single GPU. Notwithstanding the limitations, the advantages of MOPSO/D-Net make it to be a promising manner revealed by the observations.

4.3. Experiments on CIFAR-10 dataset

Experimental simulations are also carried out on the CIFAR-10 dataset, which contains 10 classes of 32×32 RGB images (Krizhevsky, 2009). In CIFAR-10, there are 50,000 training images and 10,000 testing images in total. We further split the training set into a training set and a validation set with a ratio of 9:1.

Similar to the MNIST experiment, K is first set to $2^3 = 8$ for rapid evaluations around the evolution. Four candidate values ($L \in \{3, 4, 5, 6\}$) for L are also investigated on CIFAR-10. Each convolutional block is trained by Adam optimizer with $\beta_1 = 0.9$, $\beta_2 = 0.999$ and $\varepsilon = 10^{-8}$, whereas the training requires 12 epochs. The learning rate is initialized as 0.01, and reduced by a factor of 0.2 every 5 epochs. The search algorithm terminates at 50 generations and other settings are the same as those of the MNIST experiment.

Fig. 8 shows the obtained Pareto optimal solutions of the formulated multiobjective NAS. Also similar to the observations of MNIST, each picture indicates that the final solutions are more likely to approximate the ideal point in comparison with the early solutions. The boundary solutions have been well preserved through MOPSO/D-Net. Next, the one among them with the best classification performance is picked out for each case. The optimal convolutional blocks selected are all exhibited in Fig. 9.

To further improve the model accuracy, we use $K = 256$ and stack the entire architecture following the graph as shown in Fig. 10. The structure consists of 4 blocks, 3 reduction cells, and several skip connections. The reduction cell between two blocks is used for down-sampling, which contains 1×1 convolution followed by a 2×2 average pooling layer. The skip connections are the projection shortcuts introduced in He et al. (2016). Then, we follow He et al. (2016) and augment each training data: padding 4 pixels on each side, and randomly cropping 32×32 image from the padded one or its horizontal flip. The full network is trained by 80 epochs with a learning rate of 0.1, 20 epochs with a learning rate of 0.01, 15 epochs with a learning rate of 0.001 and 5 epochs with a learning rate of 0.0001. We use SGD as the optimizer with a momentum of 0.9 and weight decay of 0.0001. The weights and biases are initialized following MSRA initialization (He, Zhang, Ren, & Sun, 2015).

Fig. 11 further shows the performances of four CNNs obtained by the proposed connection style. We observe that the auto-generated CNN of the case $L = 6$ gains the best accuracy on

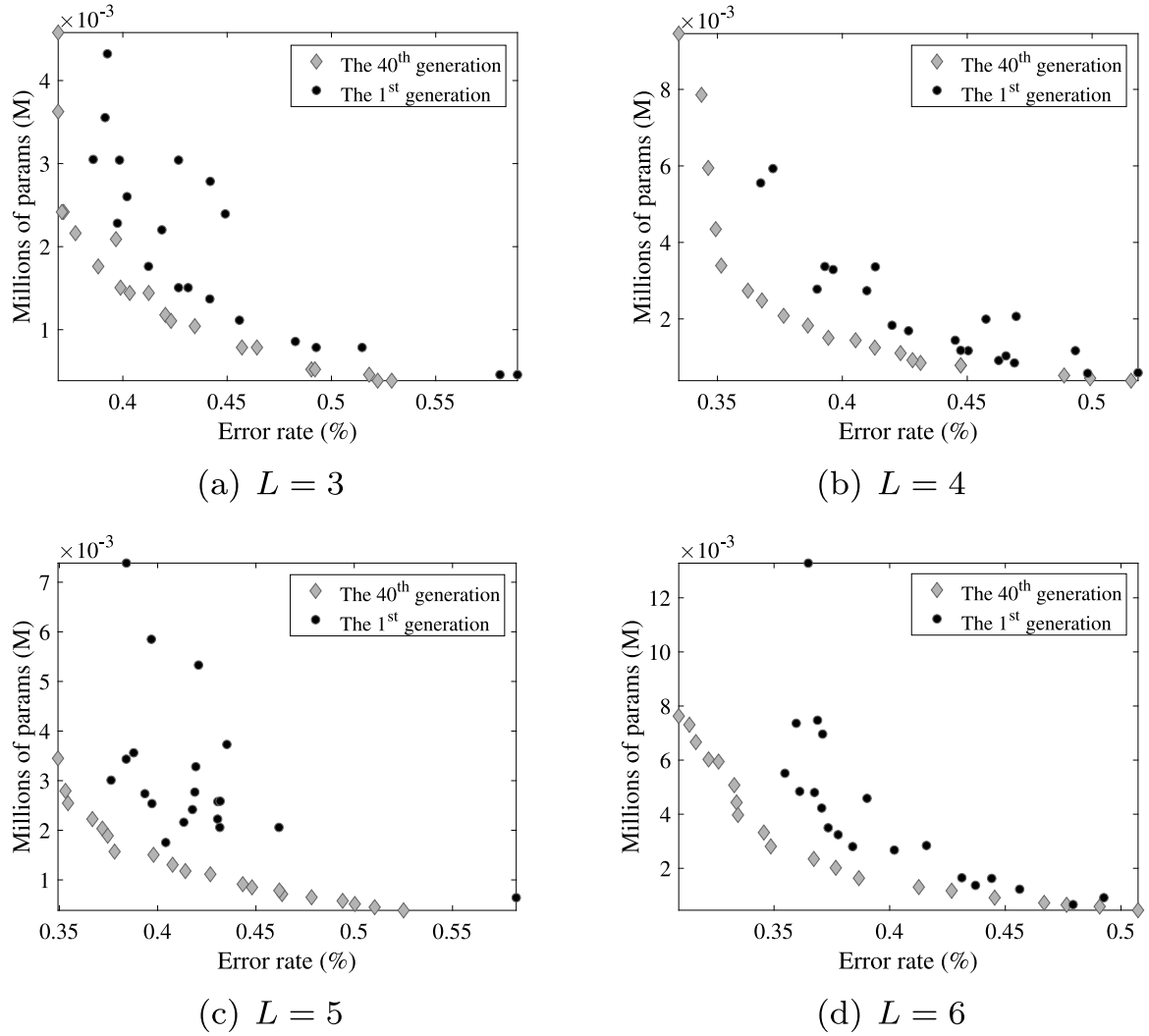


Fig. 8. The initial and final particles obtained by MOPSO/D-Net with the CIFAR-10 dataset.

Table 3

Results comparison among the proposed MOPSO/D-Net and peer algorithms on the CIFAR-10 dataset.

	Search manner	Error rate (%)	Params (M)	Inference latency (ms)	Search cost
VGGNet (Simonyan & Zisserman, 2015)	Manual	7.25	–	–	–
ResNet (depth = 110) (He et al., 2016)		6.43	1.7	–	–
DenseNet (depth = 100) (Huang et al., 2017)		4.10	7.0	–	–
MetaQNN (Baker et al., 2017)	Automated (RL)	6.92	11.2	–	10 GPUs / 8 days
NASNet (Zoph et al., 2018)		4.47	7.1	–	500 GPUs / 33 days
BlockQNN (Zhong et al., 2018)		3.57	3.9	–	1 GPU / 20 hours
MnasNet (Tan et al., 2019)		3.85	3.5	606	1 GPU / 8.2 hours
GeNet (Xie & Yuille, 2017)	Automated (EA)	7.10	–	–	10 GPUs / 17 days
CoDeepNEAT (Miikkulainen et al., 2017)		7.30	–	–	–
Hierarchical Representation (Liu et al., 2018)		3.63	–	–	200 GPUs / 1.5 days
Large-scale Evolution (Real et al., 2017)		5.4	5.4	–	– / 250 hours
MOPSO/D-Net ($K = 256$)		5.88	8.1	886	1 GPU / 8 hours

† Inference latencies are given by conducting the learned models on our experimental environment while search costs are cited from the original references, respectively.

§ All the indices of MnasNet are obtained by reproducing the algorithm.

CIFAR-10 (5.88% testing error and 8.1M params). Therefore, we pick it as the final result of MOPSO/D-Net, and compare the network with various manual and automated models in Table 3. From the table, it can be observed that the proposed algorithm achieves a better error rate than two of all the manually designed models, namely ResNet and VGGNet. The network architecture found by MOPSO/D-Net shares some similarities with two baseline models, as it also contains *conv-pool* sequences and residual

connections. Compared with NAS methods, MOPSO/D-Net also outperforms one of four RL based and two of four EA based automated models. Its error rate is 1.04% lower than MetaQNN, 1.22% lower than GeNet and 1.42% lower than CoDeepNEAT. Despite the high performance, the proposed MOPSO/D-Net requires the lowest search cost among all the peer algorithms. It significantly reduces the cost down to only 1 GPU/8 hours, where most NAS approaches require multiple GPU devices and more than one day

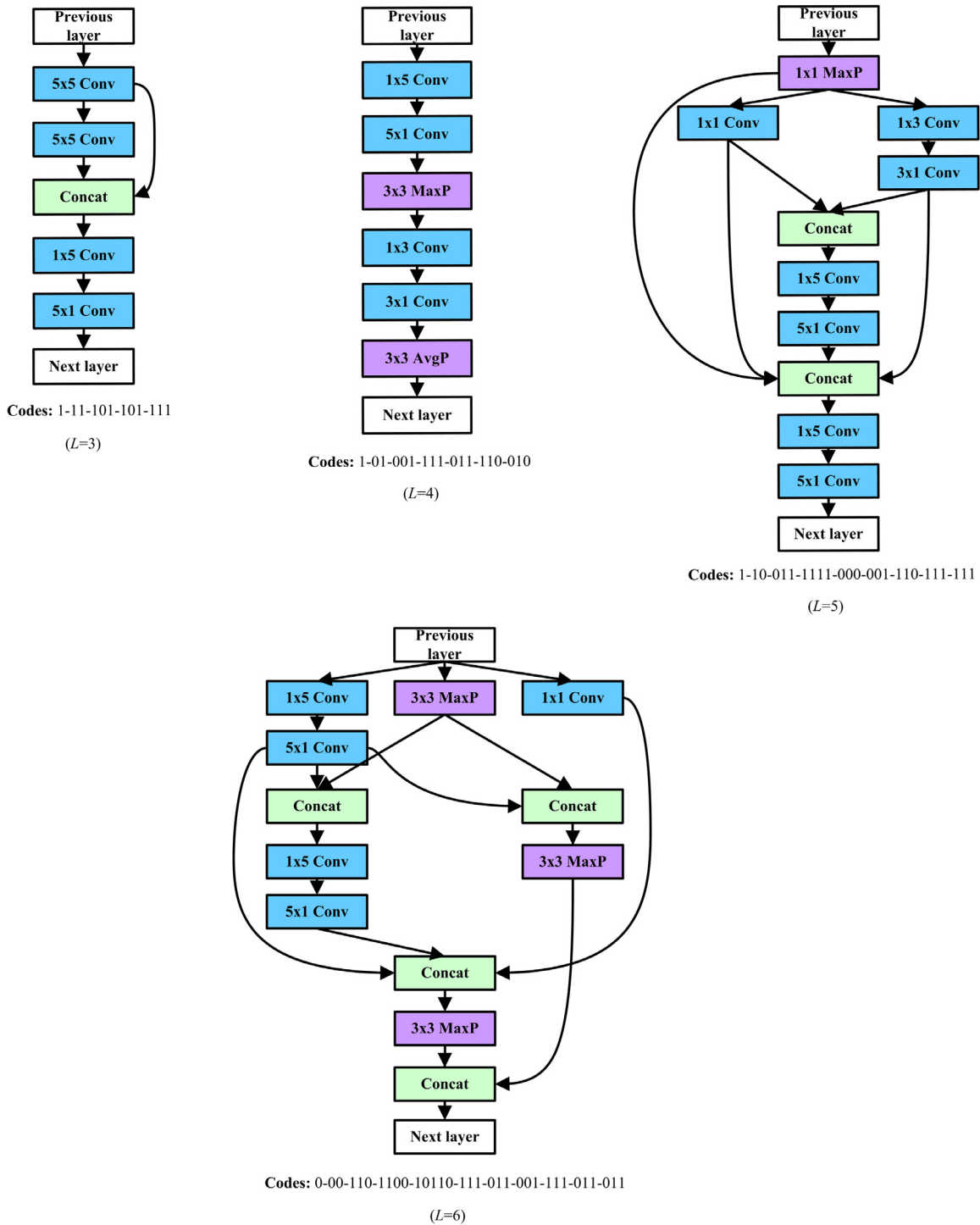


Fig. 9. The selected convolutional blocks with the CIFAR-10 dataset.

cost. This comparison verifies the effectiveness of the proposed algorithm on the CIFAR-10 dataset.

It is also worth noting that the performance of MOPSO/D-Net is worse than 3 automatically designed models found by reinforcement learning. One of possible explanations for this might be that RL based methods provide variable-length architecture space (Tan et al., 2019; Zhong et al., 2018; Zoph et al., 2018), which is more flexible than ours. Since the length of particles is fixed throughout the evolution, the maximum number of nodes, i.e. the value of L , should be predefined. Although the hybrid

binary network representation can remove the node without any connection, it still suffers from an inappropriate architecture space incurred by a fixed L . This fact potentially affects the performance of CNN discovered. On the other hand, the references (Real et al., 2017; Zoph et al., 2018) that suggest finding entire architectures also have better results than MOPSO/D-Net. They achieve such impressive performances at the expense of the computation costs, which are caused by time-consuming fitness or reward evaluations. Particularly, although the work (Liu et al., 2018) expects to search out the components with multi-level

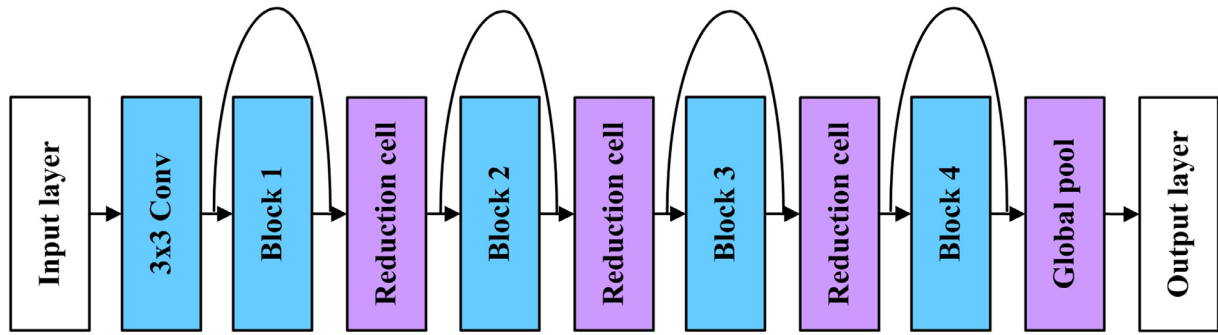


Fig. 10. The connection style of the entire architecture for the CIFAR-10 dataset.

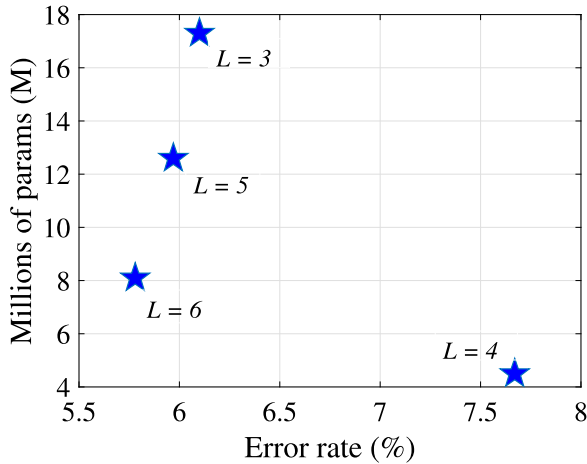


Fig. 11. The comparison among CNNs with different architectures on the CIFAR-10 dataset.

motifs, its main methodology seems close to those of algorithms aiming at learning full topology. It thereby undergoes lower error rate with considerable search cost (200 GPUs/1.5 days) in contrast with MOPSO/D-Net. Consequently, combining with a variable-length-particle update scheme and more reasonable network representation may improve results, which we will study in the future.

5. Conclusions

To realize the automating of network generation, this paper proposed a NAS method via multiobjective particle swarm optimization based on decomposition. Two conflicting goals, including the validation accuracy and params, were considered in the proposed approach where a high-performance and compact CNN is finally obtained. To achieve better implementation, we applied a hybrid binary encoding for representing the network connectivity and component layers. Besides, an adaptive PBI strategy is presented for overcoming the drawbacks of the fixed penalty. The CNN found shared similar architectures with some manually designed networks and finally achieved competitive results compared to both handcrafted and auto-generated CNNs. The work follows the general construction philosophy from exploring a basic cell to stacking a complex one and imitates the design process of human experts. Supported by parallel computing, the computation cost of NAS has been reduced to an acceptable level, which makes the approach become available and promising.

Acknowledgments

This work was supported by the National Natural Science Foundation of China [Nos. 61976108 and 61572241], the National Key R&D Program of China [No. 2017YFC0806600], the Foundation of the Peak of Six Talents of Jiangsu Province [No. 2015-DZXX-024], and the Fifth “333 High Level Talented Person Cultivating Project” of Jiangsu Province [No. (2016) III-0845].

References

- Baker, B., Gupta, O., Naik, N., & Raskar, R. (2017). Designing neural network architectures using reinforcement learning. In *International conference on learning representations*.
- Chen, T., Rubanova, Y., Bettencourt, J., & Duvenaud, D. (2018). Neural ordinary differential equations. In *International conference on neural information processing systems*.
- Dai, C., Wang, Y., & Ye, M. (2015). A new multi-objective particle swarm optimization algorithm based on decomposition. *Information Sciences*, 325, 541–557.
- Elsken, T., Hendrik, J., & Hutter, F. (2018). Neural architecture search: A survey. *arXiv:1808.05377*, arXiv preprint.
- Elsken, T., Hendrik, J., & Hutter, F. (2019). Efficient multi-objective neural architecture search via lamarckian evolution. In *International conference on learning representations*. (pp. 1–23).
- Han, F., Jiang, J., Ling, Q., & Su, B. (2019). A survey on metaheuristic optimization for random single-hidden layer feedforward neural network. *Neurocomputing*, 335, 261–273.
- He, K., Zhang, X., Ren, S., & Sun, J. (2015). Delving deep into rectifiers: Surpassing human-level performance on ImageNet classification. In *IEEE international conference on computer vision*. (pp. 1026–1034).
- He, K., Zhang, X., Ren, S., & Sun, J. (2016). Deep residual learning for image recognition. In *IEEE international conference on computer vision and pattern recognition*. (pp. 770–778).
- Hsu, C., Chang, S., Juan, D., Pan, J., Chen, Y., Wei, W., et al. (2018). MONAS: Multi-objective neural architecture search using reinforcement learning. In *JMLR: workshop and conference proceedings*. (pp. 1–8).
- Hu, J., Shen, L., Albanie, S., Sun, G., & Wu, E. (2019). Squeeze-and-excitation networks. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, <http://dx.doi.org/10.1109/TPAMI.2019.2913372>.
- Huang, G., Liu, Z., Maaten, L., & Weinberger, K. (2017). Densely connected convolutional networks. In *IEEE international conference on computer vision and pattern recognition*. (pp. 2261–2269).
- Junior, F., & Yen, G. (2019). Particle swarm optimization of deep neural networks architectures for image classification. *Swarm and Evolutionary Computation*, 49, 62–74.
- Kennedy, J., & Eberhart, R. (1995). Particle swarm optimization. In *IEEE international conference on neural networks*. (vol. 4). (pp. 1942–1948).
- Krizhevsky, A. (2009). *Learning multiple layers of features from tiny images: Technical report*, University of Toronto.
- Krizhevsky, A., Sutskever, I., & Hinton, G. (2012). ImageNet classification with deep convolutional neural networks. In *International conference on neural information processing systems*. (pp. 1097–1105).
- Lecun, Y., Bengio, Y., & Hinton, G. (2015). Deep learning. *Nature*, 521(7553), 436–444.
- LeCun, Y., Bottou, L., Bengio, Y., & Haffner, P. (1998). Gradient-based learning applied to document recognition. *Proceedings of the IEEE*, 86(11), 2278–2324.

- Lee, C., Gallagher, P., & Tu, Z. (2013). Regularization of neural networks using DropConnect. In *International conference on machine learning*. (pp. 2095–2103).
- Liu, H., Simonyan, K., Vinyals, O., Fernando, C., & Kavukcuoglu, K. (2018). Hierarchical representations for efficient architecture search. In *International conference on learning representations*.
- Miikkulainen, R., Liang, J., Meyerson, E., Rawal, A., Dan, F., Francon, O., et al. (2017). Evolving deep neural networks. [arXiv:1703.00548](#), arXiv.preprint.
- Ming, M., Wang, R., Zha, Y., & Zhang, T. (2017). Pareto adaptive penalty-based boundary intersection method for multi-objective optimization. *Information Sciences*, 414, 158–174.
- Mukhopadhyay, A., Maulik, U., Bandyopadhyay, S., & Coello Coello, C. (2014). A survey of multiobjective evolutionary algorithms for data mining: Part I. *IEEE Transactions on Evolutionary Computation*, 18(1), 4–19.
- Pham, H., Guan, Y., & Zoph, B. (2018). Efficient neural architecture search via parameter sharing. In *International conference on machine learning*.
- Qi, Y., Ma, X., Liu, F., Jiao, L., Sun, J., & Wu, J. (2014). MOEA/D with adaptive weight adjustment. *Evolutionary Computations*, 22(2), 231–264.
- Real, E., Moore, S., Selle, A., Saxena, S., Suematsu, Y., Tan, J., et al. (2017). Large-scale evolution of image classifiers. In *International conference on machine learning*.
- Sabour, S., Frosst, N., & Hinton, G. (2017). Dynamic routing between capsules. In *International conference on neural information processing systems*.
- Sainath, T., Kingsbury, B., Saon, G., Soltau, H., Mohamed, A., Dahl, G., et al. (2015). Deep convolutional neural networks for large-scale speech tasks. *Neural Networks*, 64, 39–48.
- Schmidhuber, J. (2015). Deep learning in neural networks: An overview. *Neural Networks*, 61, 85–117.
- Sheng, X., Jiang, S., & Jiang, Y. (2017). Improving the multiobjective evolutionary algorithm based on decomposition with new penalty schemes. *Soft Computing*, 21(16), 4677–4691.
- Simonyan, K., & Zisserman, A. (2015). Very deep convolutional networks for large-scale image recognition. In *International conference on learning representations*.
- Stanley, K., & Miikkulainen, R. (2002). Evolving neural networks through augmenting topologies. *Evolutionary Computation*, 10(2), 99–127.
- Szegedy, C., Ioffe, S., & Vanhoucke, V. (2016). Rethinking the inception architecture for computer vision. In *IEEE international conference on computer vision and pattern recognition*.
- Szegedy, C., Liu, W., Jia, Y., Sermanet, P., Reed, S., Anguelov, D., et al. Going deeper with convolutions. In *IEEE international conference on computer vision and pattern recognition*. (pp. 1–9).
- Tan, M., Chen, B., Pang, R., Vasudevan, V., Sandler, M., Howard, A., et al. (2019). MnasNet: Platform-aware neural architecture search for mobile. In *IEEE international conference on computer vision and pattern recognition*.
- Wang, B., Sun, Y., Xue, B., & Zhang, M. (2018). Evolving deep convolutional neural networks by variable-length particle swarm optimization for image classification. In *IEEE congress on evolutionary computation*.
- Xie, L., & Yuille, A. (2017). Genetic CNN. In *IEEE international conference on computer vision*.
- Zhang, Q., & Li, H. (2007). MOEA/D: A multiobjective evolutionary algorithm based on decomposition. *IEEE Transactions on Evolutionary Computation*, 11(6), 712–731.
- Zhong, Z., Yan, J., Wu, W., Shao, J., & Liu, C. (2018). Practical block-wise neural network architecture generation. In *IEEE international conference on computer vision and pattern recognition*.
- Zoph, B., & Le, Q. (2017). Neural architecture search with reinforcement learning. In *IEEE international conference on learning representations*.
- Zoph, B., Vasudevan, V., Shlens, J., & Le, Q. (2018). Learning transferable architectures for scalable image recognition. In *International conference on computer vision and pattern recognition*.